FB15k-237 Link Prediction: Baseline Training and Preliminary Experiments

Thalia Delhaise (66937), Lenny Briclet (66926), André Fonseca (58225), Rafael Gufler (66081)

Project 425282 - Report for Milestone 1 (Deep Learning)

Abstract

We build the multi-relational **FB15k-237** graph, train classical knowledge graph embedding baselines **TransE** and **RotatE** under matched hyperparameters and a **filtered** ranking protocol, and report preliminary **MRR / Hits@k** with training curves. Our central question is: *do relation-structural patterns (symmetry, composition, inversion) explain where RotatE gains over TransE, and does custom hard negative mining amplify those gains?* This report covers the **data pipeline**, the **baseline** setup, and a concrete plan for Milestone 2: custom hard-negative sampling (standalone PyTorch), relation- and degree-wise slice metrics, and qualitative error analysis. Code: `code/train_baseline_kge.py` in the repository.

1. Introduction and plan

Goal: given training triples (h, r, t) in a knowledge graph, rank the correct missing entity for queries $(h, r, ?)$ and $(?, r, t)$. We compare TransE to the stronger RotatE [3] on **identical** train/val/test partitions and the same **filtered** evaluation (true triples in any split are not treated as false tails/heads) per Bordes et al. [2]. The central scientific question is: *do relation-structural patterns—symmetry, composition, inversion—explain where RotatE gains over TransE on FB15k-237, and does custom hard negative mining amplify or redistribute those gains?* This sharpens a pure benchmark comparison into an interpretable experimental claim about *where* improvements appear (aggregate vs. relation- or degree-specific regimes). Our own contribution beyond PyKEEN consists of: (i) a standalone **PyTorch** hard-negative sampler, (ii) automated **slice evaluation** by relation and entity degree, and (iii) a structured **error analysis**. Milestone 1 establishes: **data ready**, a **baseline** running end-to-end, **preliminary** metrics and curves, and a clear **next-step** plan.

2. Problem statement, dataset, and evaluation

Dataset. FB15k-237: 14,541 entities, 237 relations, 272,115 / 17,535 / 20,466 train, validation, and test triples (official split) [3]. The graph is $\mathcal{G} = (\mathcal{E}, \mathcal{R}, \mathcal{T})$, $\mathcal{T} \subseteq \mathcal{E} \times \mathcal{R} \times \mathcal{E}$. No extra subsampling.

Metrics. Filtered mean reciprocal rank (MRR) and Hits@k ($k \in \{1, 3, 10\}$) on head and tail prediction, then averaged, as in standard benchmarks.

Expected. A controlled comparison under matched hyperparameters: same embedding size d, same optimizer family, same maximum epochs (with early stopping on val MRR), and same number of corruptions per positive where applicable.

3. Data pipeline and preprocessing

Download. The benchmark is loaded via **PyKEEN** [1] (dataset name `fb15k237`, cached locally on first use). The library maps each entity and relation to integer IDs and materializes a `TriplesFactory` for train, validation, and test.

Check. We verify triple counts and disjointness of test from train. The training script computes and logs dataset **statistics**: relation-frequency histogram and entity **in-/out-degree** distribution (saved to `artifacts/baseline/dataset_stats.jso`
Milestone 2 will include explicit examples of one-to-one, one-to-many, many-to-one, and many-to-many relation patterns as direct motivation for the slice evaluation and error analysis.

4. Baseline models and training setup (PyKEEN)

We start with two classical models with fixed scoring: TransE and RotatE [3]. The rest of this section is the **configuration we actually run** (edit `train_baseline_kge.py` if you change it).

Model	TransE, RotatE (separate runs)
Formulation	Entity embeddings in $\mathbb{R}^d$; score triples; rank tails/heads.
Input / shape	Mini-batch of triple indices; shape `[batch, 3]`.
Negatives	Corrupt head or tail; Bernoulli as in KGE default practice.
Loss	Margin / pairwise ranking (default per model in PyKEEN).
Optimiser	Adam ($\beta_1 = 0.9$, $\beta_2 = 0.999$); LR 10^{-3}.
Hyperparam.	$d = 128$; up to 15 epochs (preliminary); `batch_size`=1024; LR 10^{-3}; early stop if val MRR flat 3 epochs. Full runs with 50 epochs and patience 10 are planned for Milestone 2.
Seed / env.	`random_seed=42`; Python 3, PyTorch, PyKEEN; GPU optional.
Time budget	TransE: MPS in our run (Apple M-series). RotatE: CUDA in our run; MPS is unavailable for RotatE's complex ops (CPU if no GPU).

The script writes `artifacts/baseline/<Model>_summary.txt` and a PyKEEN result folder; copy key metrics and loss curves from the run into the table and figure above.

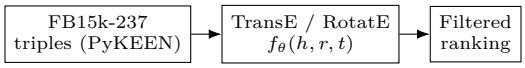

Figure 1: Baseline KGE data flow for Milestone 1. Later work adds a custom hard-negative sampler (standalone PyTorch), relation- and degree-wise slice evaluation, and qualitative error analysis.

5. Preliminary results and training curves

Table 1: Preliminary filtered metrics on the test set (realistic ranking, `both` averaged; 15 epochs max, early stopping with patience 3 on val MRR; $d = 128$, Adam, lr=10^{-3}, seed=42).

Model	MRR	H@1	H@3	H@10	Notes
TransE	0.129	0.057	0.152	0.263	15 ep. max, bs=1024, MPS (439s train+eval)
RotatE	**0.234**	**0.159**	**0.263**	**0.378**	15 ep. max, bs=1024, CUDA (169s train+eval)

RotatE consistently outperforms TransE across all metrics at matched hyperparameters: MRR improves from 0.129 to 0.234 (+81%), and H@10 from 0.263 to 0.378. Both models were still improving at epoch 15 (no early stopping triggered), suggesting further gains with longer training in Milestone 2. The asymmetry between head prediction (RotatE H@10=0.261) and tail prediction (RotatE H@10=0.494) motivates the planned relation-type slice analysis.

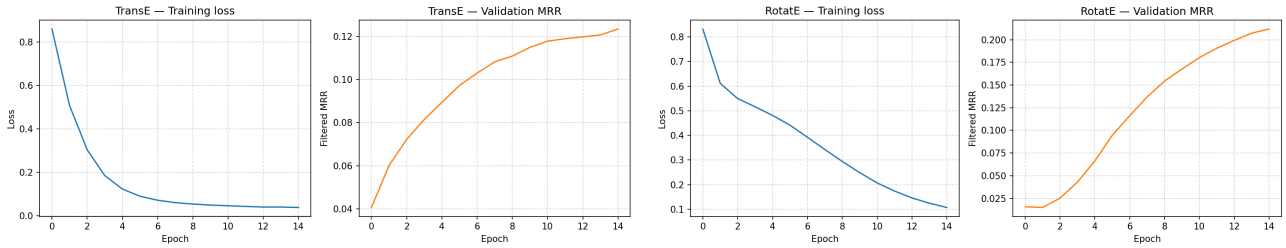

Figure 2: Training loss and validation MRR vs. epoch for TransE (left, MPS) and RotatE (right, GPU). Both models continue to improve at epoch 15, motivating longer runs in Milestone 2.

6. Known issues and next steps

Current issues. (1) Final hyperparameters may need tuning after full runs complete. (2) Negative sampling is currently default uniform/Bernoulli via PyKEEN; **custom** hard negative mining (self-adversarial or frequency-weighted, implemented as standalone **PyTorch** code) is the central group contribution and is planned for Milestone 2. (3) Slice evaluation and error analysis are not yet automated.

Milestone 2 and project plan. (a) **Custom hard negative mining**: implement self-adversarial sampling [3] in standalone PyTorch and ablate against the Bernoulli baseline. (b) Relation- and degree-**slice** metrics to directly answer the central question about structural patterns. (c) **Error analysis**: sample successes and failures by relation type (symmetric, compositional, inverse) and entity degree. (d) **Ablation** on embedding dimension d and training budget as a compute-vs.-accuracy trade-off. (e) Short discussion of wall-clock time and practical trade-offs. Code organisation: one script per experiment family; one notebook for consolidated tables and plots.

Milestone 1 deliverables checklist. ✓ Data available via PyKEEN; ✓ ID mappings; ✓ baseline **model + architecture + loss + optim + budget** documented; ✓ skeleton + runnable training script; ✓ preliminary **results** and training curves; ✓ this report.

Code repository. Baseline training: `code/train_baseline_kge.py`. Unify with KG course: same dataset and protocol.

References

[1] Mehdi Ali, Max Berrendorf, Charles Tapley Hoyt, Laurent Vermue, et al. PyKEEN 1.0: A python library for training and evaluating knowledge graph embeddings. *JMLR*, 22(82):1–6, 2021.

[2] Antoine Bordes, Nicolas Usunier, Alberto Garcia-Durán, Jason Weston, and Oksana Yakhnenko. Translating embeddings for modeling multi-relational data. In *NeurIPS*, 2013.

[3] Zhiqing Sun, Zhi-Hong Deng, Jian-Yun Nie, and Jian Tang. RotatE: Knowledge graph embedding by relational rotation in complex space. In *ICLR*, 2019.